VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DISCRETE STRUCTURE FOR COMPUTING - CO1007

ASSIGNMENT REPORT

TRAVELLING SALESMAN PROBLEM

Class: CC02

Student: Phm Vũ Khôi Nguyên - 2412372

HO CHI MINH CITY, JUNE 2025

Contents

1 Introduction to the Traveling Salesman Problem **2**

2 Problem Requirements **2**
2.1 Specification . . . 2
2.1.1 Input . . . 2
2.1.2 Output . . . 2

3 Solution Approaches **3**
3.1 Support Functions . . . 3
3.1.1 `getVertices(edgeList, numEdges)` . . . 3
3.1.2 `getVertexIndex(vertices, c)` . . . 4
3.2 Bellman-Held-Karp Algorithm for Small Instances . . . 4
3.2.1 Algorithm Explanation . . . 4
3.2.2 Code Implementation . . . 6
3.3 Ant Colony Optimization for Large Instances . . . 8
3.3.1 Algorithm Explanation . . . 8
3.3.2 Code Implementation . . . 9

4 Analysis and Comparison **11**

5 Conclusion **11**

6 References **12**

1 Introduction to the Traveling Salesman Problem

The Traveling Salesman Problem (TSP) is a well-known combinatorial optimization problem in computer science and operations research. Given a set of cities and the distances between each pair, the goal is to find the shortest possible route that visits each city exactly once and returns to the starting point. Classified as NP-hard, solving TSP exactly becomes computationally impractical for large instances, necessitating a combination of exact and approximate methods tailored to problem size.

This report outlines my solution implemented in the `Traveling()` function, which employs two distinct algorithms based on the number of vertices. For small instances (up to 20 vertices), I use the Bellman-Held-Karp dynamic programming algorithm to guarantee an optimal solution. For larger instances (more than 20 vertices), I apply the Ant Colony Optimization (ACO) metaheuristic to efficiently compute a near-optimal solution. This hybrid approach balances accuracy and scalability, addressing the diverse requirements of the TSP across different graph sizes.

2 Problem Requirements

2.1 Specification

The assignment tasks students with implementing the `Traveling()` function to determine the shortest Hamiltonian cycle in a directed weighted graph, as specified in the assignment document.

2.1.1 Input

- `edgeList (Graph)`: A 2D array of integers where each row `[u, v, w]` represents an edge from vertex `u` to vertex `v` with weight `w`. Vertices are characters (e.g., 'A' to 'Z'), and weights are positive integers.
- `numEdges`: The total number of edges in the graph.
- `startVertex`: The starting vertex, denoted as a character (e.g., 'A').

2.1.2 Output

- A string representing the shortest path that visits each vertex exactly once and returns to `startVertex`, with vertices separated by spaces (e.g., "A B C A"). If no valid tour exists, an empty string is returned.

3 Solution Approaches

My solution adapts its strategy based on the graph size, using Bellman-Held-Karp for smaller instances and ACO for larger ones. Below, I detail each approach, explaining the algorithms and their implementations step-by-step, closely tied to my code.

3.1 Support Functions

Before diving into the main algorithms, it's essential to understand the support functions used in the implementation.

3.1.1 `getVertices(edgeList, numEdges)`

This function extracts unique vertices from the given `edgeList` and returns them in a sorted vector.

Example: Suppose `edgeList` contains:

A, B, 2

B, C, 1

C, A, 6

Then, `getVertices` will return `['A', 'B', 'C']`.

Code:

```
vector<char> getVertices(int edgeList[][3], int numEdges) {
    bool appeared[1000] = {false};
    vector<char> vertices;
    for (int i = 0; i < numEdges; i++) {
        char u = (char)edgeList[i][0];
        char v = (char)edgeList[i][1];
        if (!appeared[(int)u]) {
            appeared[(int)u] = true;
            vertices.push_back(u);
        }
        if (!appeared[(int)v]) {
            appeared[(int)v] = true;
            vertices.push_back(v);
        }
    }
    sort(vertices.begin(), vertices.end());
    return vertices;
}
```

3.1.2 `getVertexIndex(vertices, c)`

This function returns the index of vertex `c` in the sorted vector `vertices`.

Example: For `vertices = ['A', 'B', 'C']`, `getVertexIndex(vertices, 'B')` returns 1.

Code:

```
int getVertexIndex(const vector<char>& vertices, char c) {
    for (int i = 0; i < vertices.size(); i++) {
        if (vertices[i] == c) {
            return i;
        }
    }
    return -1; // Not found
}
```

3.2 Bellman-Held-Karp Algorithm for Small Instances

For graphs with up to 20 vertices, I implement the Bellman-Held-Karp algorithm, an exact dynamic programming method that ensures the optimal TSP solution.

3.2.1 Algorithm Explanation

This algorithm uses a dynamic programming table `dp[mask][u]`, where `mask` is a bitmask indicating the subset of visited vertices, and `u` is the last vertex in the path. The value `dp[mask][u]` represents the minimum cost to reach `u` from the starting vertex, visiting all vertices in `mask` exactly once. A `prev[mask][u]` table tracks the previous vertex for path reconstruction.

Example: Consider a graph with three vertices: A, B, and C, with the following directed edges and weights:

- A $\rightarrow$ B: 2
- A $\rightarrow$ C: 5
- B $\rightarrow$ C: 1
- C $\rightarrow$ B: 3
- B $\rightarrow$ A: 4
- C $\rightarrow$ A: 6

Starting from A, the optimal tour is A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ A with a total cost of $2 + 1 + 6 = 9$.

Here's how the algorithm computes this:

1. **Initialization:**
   - Map vertices to indices: $A = 0$, $B = 1$, $C = 2$.
   - Build the weight matrix:

$$\text{weight} = \begin{bmatrix} -1 & 2 & 5 \\ 4 & -1 & 1 \\ 6 & 3 & -1 \end{bmatrix}$$

   - Initialize the DP table with `dp[1][0] = 0`, where mask 1 (binary 001) represents only vertex A.
2. **Filling the DP table:**
   - For mask=1 (001), $u = 0$ (A):
     - To $v = 1$ (B): nextMask = $001 \mid 010 = 011$ (3), `dp[3][1] = 0 + 2 = 2`
     - To $v = 2$ (C): nextMask = $001 \mid 100 = 101$ (5), `dp[5][2] = 0 + 5 = 5`
   - For mask=3 (011), $u = 1$ (B):
     - To $v = 2$ (C): nextMask = $011 \mid 100 = 111$ (7), `dp[7][2] = 2 + 1 = 3`
   - For mask=5 (101), $u = 2$ (C):
     - To $v = 1$ (B): nextMask = $101 \mid 010 = 111$ (7), `dp[7][1] = 5 + 3 = 8`
3. **Tour Completion:**
   - For fullMask=7 (111):
     - For $u = 1$ (B): `dp[7][1] = 8`, return cost = 4, total = 12
     - For $u = 2$ (C): `dp[7][2] = 3`, return cost = 6, total = 9
   - Choose $u = 2$ with minCost=9.
4. **Path Reconstruction:**
   - From last=2 (C), mask=7, `prev[7][2] = 1` (B), then mask = $7 \oplus 4 = 3$
   - From $u = 1$ (B), mask=3, `prev[3][1] = 0` (A)
   - Path: $A \to B \to C \to A$

3.2.2 Code Implementation

The implementation is broken down into steps:

Step 1: Extract and sort unique vertices

```
vector<char> vertices = getVertices(edgeList, numEdges);
int numVertices = vertices.size();
int startIndex = getVertexIndex(vertices, startVertex);
```

Step 2: Build weight matrix

```
vector<vector<int>> weight(numVertices, vector<int>(numVertices,
    -1));
for (int i = 0; i < numEdges; i++) {
    int u = getVertexIndex(vertices, (char)edgeList[i][0]);
    int v = getVertexIndex(vertices, (char)edgeList[i][1]);
    weight[u][v] = edgeList[i][2];
}
```

Step 3: Initialize DP tables

```
int subsetCount = 1 << numVertices;
vector<vector<int>> dp(subsetCount, vector<int>(numVertices, -1));
vector<vector<int>> prev(subsetCount, vector<int>(numVertices, -1))
    ;
dp[1 << startIndex][startIndex] = 0;
```

Step 4: Fill DP table

```
for (int mask = 0; mask < subsetCount; mask++) {
    for (int u = 0; u < numVertices; u++) {
        if (dp[mask][u] == -1) continue;
        for (int v = 0; v < numVertices; v++) {
            if ((mask >> v) & 1 || weight[u][v] == -1) continue;
            int nextMask = mask | (1 << v);
            int newDist = dp[mask][u] + weight[u][v];
            if (dp[nextMask][v] == -1 || newDist < dp[nextMask][v])
                {
                dp[nextMask][v] = newDist;
                prev[nextMask][v] = u;
            }
        }
    }
}
```

Step 5: Find optimal last vertex

```
int fullMask = (1 << numVertices) - 1;
int last = -1;
int minCost = -1;
for (int u = 0; u < numVertices; u++) {
    if (u == startIndex || dp[fullMask][u] == -1 || weight[u][
        startIndex] == -1) continue;
    int cost = dp[fullMask][u] + weight[u][startIndex];
    if (minCost == -1 || cost < minCost) {
```

```
            minCost = cost;
            last = u;
        }
    }
```

Step 6: Reconstruct the path

```
vector<int> path;
int mask = fullMask;
int current = last;
while (current != -1) {
    path.push_back(current);
    int temp = prev[mask][current];
    mask ^= (1 << current);
    current = temp;
}
reverse(path.begin(), path.end());
```

Step 7: Format output string

```
string result = "";
for (int i = 0; i < path.size(); i++) {
    result += vertices[path[i]];
    if (i < path.size()) result += "␣";
}
result += startVertex;
```

3.3 Ant Colony Optimization for Large Instances

For graphs exceeding 20 vertices, I use ACO, a metaheuristic inspired by the pheromone-based navigation of ants, suitable for finding near-optimal solutions efficiently.

3.3.1 Algorithm Explanation

Ant Colony Optimization (ACO) is inspired by how ants find shortest paths by depositing pheromones on trails. Each ant builds a tour by probabilistically selecting cities based on pheromone levels (τ_{ij}) and heuristic values ($\eta_{ij} = \frac{1}{\text{distance}_{ij}}$), using roulette wheel selection to favor shorter or more-traveled edges. Pheromones evaporate over iterations, and ants deposit new pheromones based on tour quality, guiding the colony toward near-optimal solutions.

Example: Using the same graph with vertices A, B, and C:

- **Initial Pheromone Levels:** Set to 0.1 for all edges.
- **Heuristic Values:**
  - For edge $A \to B$: $\eta_{AB} = \frac{1}{2} = 0.5$
  - For edge $A \to C$: $\eta_{AC} = \frac{1}{5} = 0.2$
  - Similarly for other edges.
- **Parameters:** Set `ALPHA = 1`, `BETA = 1`.
- **Probability Calculation:**
  - **Probability Calculation:** The probability of choosing city j from the current city is given by:

$$P(j \mid \text{current}) = \frac{(\tau_{\text{current},j})^{\alpha} \cdot (\eta_{\text{current},j})^{\beta}}{\sum_{k \notin \text{visited}} (\tau_{\text{current},k})^{\alpha} \cdot (\eta_{\text{current},k})^{\beta}}$$

    For the example with $\alpha = 1$, $\beta = 1$:
    - Probability of choosing B from A:

$$P(B \mid A) = \frac{\tau_{AB} \cdot \eta_{AB}}{\tau_{AB} \cdot \eta_{AB} + \tau_{AC} \cdot \eta_{AC}} = \frac{0.1 \cdot 0.5}{(0.1 \cdot 0.5) + (0.1 \cdot 0.2)} = \frac{0.05}{0.05 + 0.02} \approx 0.714$$

    - Probability of choosing C from A:

$$P(C \mid A) = \frac{\tau_{AC} \cdot \eta_{AC}}{\tau_{AB} \cdot \eta_{AB} + \tau_{AC} \cdot \eta_{AC}} = \frac{0.1 \cdot 0.2}{(0.1 \cdot 0.5) + (0.1 \cdot 0.2)} = \frac{0.02}{0.05 + 0.02} \approx 0.286$$

    These probabilities are used in a **roulette wheel selection** mechanism to choose the next city.
- **Tour Construction:** An ant may construct the tour $A \to B \to C \to A$, with a total cost of 9 (calculated as $2 + 1 + 6$).
- **Pheromone Update:**
  - Parameters: `RHO = 0.1` (evaporation rate), `Q = 100` (pheromone deposit constant).
  - Deposit on tour edges: $\Delta\tau = \frac{Q}{L} = \frac{100}{9} \approx 11.11$, where L is the tour length.

3.3.2 Code Implementation

Step 1: Initialize distance matrix

```
vector<vector<int>> distance(numVertices, vector<int>(numVertices,
    0));
for (int i = 0; i < numEdges; i++) {
    int u = getVertexIndex(vertices, (char)edgeList[i][0]);
    int v = getVertexIndex(vertices, (char)edgeList[i][1]);
    distance[u][v] = edgeList[i][2];
}
```

Step 2: Initialize pheromone and heuristic matrices

```
vector<vector<double>> pheromone(numVertices, vector<double>(
    numVertices, 0.0));
vector<vector<double>> heuristic(numVertices, vector<double>(
    numVertices, 0.0));
for (int i = 0; i < numVertices; i++) {
    for (int j = 0; j < numVertices; j++) {
        if (distance[i][j] > 0) {
            pheromone[i][j] = INITIAL_PHEROMONE;
            heuristic[i][j] = 1.0 / distance[i][j];
        }
    }
}
```

Step 3: Main loop - Ants build tours

```
for (int k = 0; k < NUM_ANTS; k++) {
    vector<bool> visited(numVertices, false);
    vector<int> tour;
    tour.push_back(startIndex);
    visited[startIndex] = true;
    while (tour.size() < numVertices) {
        int current = tour.back();
        double prob_sum = 0.0;
        vector<pair<double, int>> candidates;
        // Collect unvisited cities and compute their selection
            probabilities
        for (int j = 0; j < numVertices; j++) {
            if (!visited[j] && distance[current][j] > 0) {
                // Probability = (pheromone)^ALPHA * (heuristic)^
                    BETA
                double prob = pow(pheromone[current][j], ALPHA) *
                    pow(heuristic[current][j], BETA);
                prob_sum += prob; // Sum probabilities for
                    normalization
                candidates.emplace_back(prob, j); // Store
                    probability and city index
            }
        }
        if (prob_sum > 0) {
            // Generate random number for roulette wheel selection
```

```
            double r = dis(gen) * prob_sum; // dis and gen are for
               random number generation
            double cumulative = 0.0;
            // Perform roulette wheel selection
            for (const auto& candidate : candidates) {
                cumulative += candidate.first; // Accumulate
                   probabilities
                if (cumulative >= r) { // Select city when
                   cumulative sum exceeds random number
                    tour.push_back(candidate.second);
                    visited[candidate.second] = true;
                    break;
                }
            }
        } else {
            break; // No valid cities to visit
        }
    }
    if (tour.size() == numVertices && distance[tour.back()][
       startIndex] > 0) {
        double length = 0.0;
        for (int i = 0; i < tour.size() - 1; i++) {
            length += distance[tour[i]][tour[i + 1]];
        }
        length += distance[tour.back()][startIndex];
        ant_lengths[k] = length;
        ant_tours[k] = tour;
        if (best_length < 0 || length < best_length) {
            best_length = length;
            best_tour = tour;
        }
    }
}
```

Step 4: Update pheromones

```
// Evaporate pheromones
for (int i = 0; i < numVertices; i++) {
    for (int j = 0; j < numVertices; j++) {
        if (distance[i][j] > 0) {
            pheromone[i][j] *= (1 - RHO);
        }
    }
}
// Deposit pheromones
for (int k = 0; k < NUM_ANTS; k++) {
    if (ant_lengths[k] > 0) {
        double deposit = Q / ant_lengths[k];
        const auto& tour = ant_tours[k];
        for (int i = 0; i < tour.size() - 1; i++) {
            int u = tour[i];
            int v = tour[i + 1];
```

```
                pheromone[u][v] += deposit;
            }
            pheromone[tour.back()][startIndex] += deposit;
        }
    }
```

Step 5: Format output string

```
string result = "";
for (int i = 0; i < best_tour.size(); i++) {
    result += vertices[best_tour[i]];
    if (i < best_tour.size()) result += " ";
}
result += startVertex;
```

4 Analysis and Comparison

- **Bellman-Held-Karp**:
  - *Time Complexity*: $O(2^n \cdot n^2)$, exponential and feasible only for $n \leq 20$.
  - *Space Complexity*: $O(2^n \cdot n)$, due to the DP tables.
  - *Strength*: Guarantees the optimal solution, ideal for small graphs.
  - *Limitation*: Impractical for large n due to exponential growth.
- **Ant Colony Optimization**:
  - *Time Complexity*: $O(\text{N_EPOCH} \cdot \text{NUM_ANTS} \cdot n^2)$, polynomial and scalable.
  - *Space Complexity*: $O(n^2)$, for distance, pheromone, and heuristic matrices.
  - *Strength*: Efficient for large graphs, providing near-optimal solutions.
  - *Limitation*: Solution quality depends on parameter tuning and may not be optimal.

This hybrid strategy ensures both precision for small instances and efficiency for larger ones, meeting the assignment's diverse needs.

5 Conclusion

This report presents my solution to the TSP, integrating the exact Bellman-Held-Karp algorithm for small instances and the ACO metaheuristic for larger ones within the `Traveling()` function. By adapting the approach based on graph size, my implementation achieves a balance of optimality and computational feasibility, effectively addressing the challenges posed by the NP-hard nature of TSP.

6 References

1. Bellman, R. (1962). Dynamic Programming Treatment of the Travelling Salesman Problem. *Journal of the ACM*, 9(1), 61–63.

2. Held, M., & Karp, R. M. (1962). A Dynamic Programming Approach to Sequencing Problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1), 196–210.

3. Dorigo, M., & Gambardella, L. M. (1997). Ant Colony System: A Cooperative Learning Approach to the Traveling Salesman Problem. *IEEE Transactions on Evolutionary Computation*, 1(1), 53–66.

4. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.